

Spring Boot Interceptors — Quick Revision Notes

1. Problem Interceptors Solve

If you write the same common logic (logging, auth, timing) in every controller method, code gets duplicated and messy. Interceptor gives one central place to write this common logic.

2. Where Interceptor Fits

`DispatcherServlet` receives the request → `HandlerMapping` finds the handler → Interceptors run → then `HandlerAdapter` calls the controller.

3. Three Lifecycle Methods

`preHandle()` runs before the controller, `postHandle()` runs after the controller succeeds (before view rendering), and `afterCompletion()` runs after the full request cycle finishes.

4. Creating a Custom Interceptor

Implement the `HandlerInterceptor` interface and add `@Component`. Override only the methods you need, the rest have default implementations.

5. Method Parameters

`HttpServletRequest` = incoming request data. `HttpServletResponse` = used to set the outgoing response. `Object handler` = the selected handler (usually a `HandlerMethod`, check with `instanceof` before casting).

6. `preHandle()`

Runs before the controller is called. Return `true` to continue, return `false` to stop the request (controller won't run). If you return `false`, you must write the response yourself.

7. `postHandle()`

Runs after the controller executes successfully, before view rendering. In REST controllers, `modelAndView` is usually `null` because the response is already written, so it's not reliable for modifying REST bodies.

8. `afterCompletion()`

Like a `finally` block — runs after the entire request cycle. Used for cleanup, final logging, execution time. `ex == null` doesn't always mean "no error" (the exception may already be handled elsewhere).

9. Registering an Interceptor

Just adding `@Component` is not enough. You must implement `WebMvcConfigurer` and register it inside `addInterceptors()` using `registry.addInterceptor()`.

10. Logging Request Details

Inside the interceptor, you can get HTTP info using `request.getMethod()`, `getRequestURI()`, `getHeader()`, and get controller/method name using `handler instanceof HandlerMethod`.

11. API Key Authentication Example

In `preHandle()`, check a header (`X-API-KEY`); if it's missing or wrong, write a `401 Unauthorized` response and return `false` — this is only for learning, use Spring Security in production.

12. Authorization Example (401 vs 403)

`401` = user is not authenticated (missing/invalid token). `403` = user is authenticated but doesn't have permission (e.g., normal user trying an admin action).

13. Measuring Execution Time

Interceptors are singletons, so storing start time in an instance field is unsafe (concurrent requests will clash). Correct way: store it using `request.setAttribute()` and retrieve it in `afterCompletion()`. Use `System.nanoTime()`, not `currentTimeMillis()` (clock can be adjusted).

14. URL Patterns (Include)

`addPathPatterns("/api/students/**")` specifies which paths the interceptor applies to. `**` = nested paths, `*` = single segment.

15. Excluding URL Patterns

`excludePathPatterns()` lets you skip specific paths (like login/register) from the interceptor, even if they match the include pattern.

16. Multiple Interceptors & Order

`preHandle()` runs in forward order (A then B), `postHandle()` and `afterCompletion()` run in reverse order (B then A) — like nested function calls. Use `.order(n)` to set sequence — lower number runs first.

17. Filters vs Interceptors Flow

Filter works at the servlet layer (outside DispatcherServlet), interceptor works at the Spring MVC layer (after handler is selected). Filter can wrap the request/response, interceptor cannot.

18. Filter vs Interceptor Table

Filter = servlet-level, can wrap request/response, doesn't know the controller method.

Interceptor = MVC-level, knows the controller/method, but not suitable for wrapping the body.

19. When to Use Filter

Use a filter when the concern applies to the whole HTTP request (body wrapping, compression, CORS, encoding, security before MVC). Spring Security uses filters for this reason.

20. When to Use Interceptor

Use an interceptor when the logic depends on the Spring MVC handler (logging controller/method, checking annotations, tenant context, logic around specific endpoints).

21. Common Mistakes

- Creating an interceptor but forgetting to register it
- Returning `false` without writing a response
- Storing request data in an instance field (singleton issue)
- Casting the handler without an `instanceof` check
- Trying to modify a REST body in `postHandle()`
- Building production security manually instead of using Spring Security
- Assuming `ex == null` means no error occurred

22. Final Mental Model

Filter asks: "what should happen around the whole HTTP request?" Interceptor asks: "what should happen around the selected MVC handler?" Remember the rule: `preHandle` → before controller, `postHandle` → after success, `afterCompletion` → after everything. Multiple interceptors: forward before, reverse after.